Intro to N64 Reverse Engineering & Randomizers

•••

Explore the game design and technical details of randomizers for your favorite games!

presented by Emily & Grayson
from Sleepy Bits Games at Fire & Ice RGX 2026

I have been in the games industry since 2007, mostly focusing on graphics and optimization.
I founded Sleepy Bits Games in 2019 with my wife Emily.

All code in this presentation is open source!

- While most code will only be applicable to Shadowgate 64, the tools and concepts can apply to any game.
- No need to take notes or try to remember every detail.
- All you need is this repository:

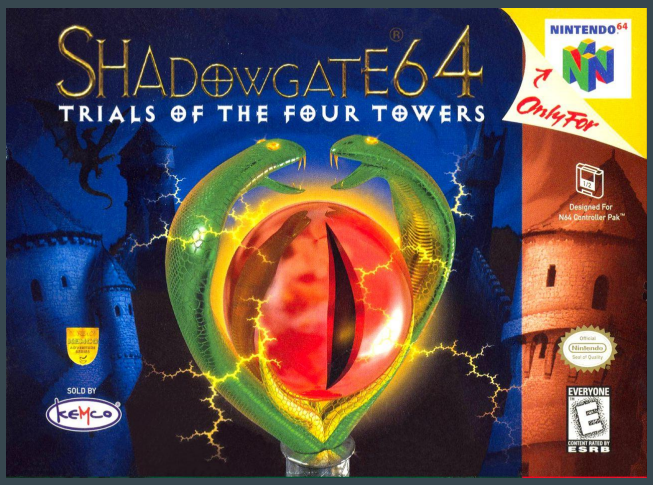
  https://github.com/grendell/sg64rando

What is a randomizer?

- Randomizers are a great way to breathe new life into your favorite video games.
- Pokémon, Mario 64, Ocarina of Time, Resident Evil, Dark Souls, and many, many other games have amazing randomizers.
- Resident Evil 2 for the N64 actually had a built-in randomizer in 1999!

Randomizers are a great way to breathe new life into your favorite video games by shuffling items, enemies, and/or doors while maintaining gameplay logic so that the game can still be completed.

Introducing Shadowgate 64: Trials of the Four Towers

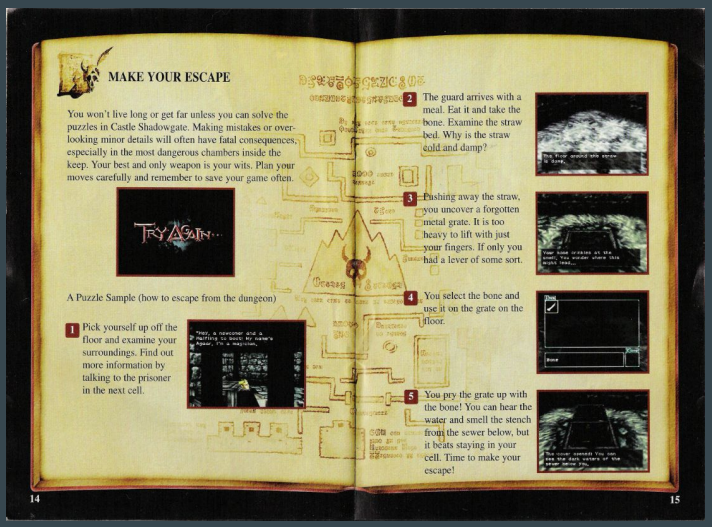


Shadowgate 64 cover art from thecoverproject.net

Shadowgate 64 was the 1999 sequel to Shadowgate, the 1987 classic, and the 1993 Beyond Shadowgate for the TurboGrafx CD, which no one played.
Confusingly, there's also a Beyond Shadowgate from 2024, which is a sequel to the 1987 game.
If you're interested in getting into the franchise, the original is available on most vintage computers, NES, Gameboy Color, and even the Playdate.
A remake was released for PC and modern consoles in 2014, but today we're going to be discussing the N64 exclusive, Shadowgate 64.

It is a 3D, first person puzzle and adventure game with a fantasy setting.
The events of the original are mentioned throughout the game, but prior knowledge isn't strictly required.

Spoiler alert!



MAKE YOUR ESCAPE

You won't live long or get far unless you can solve the puzzles in Castle Shadowgate. Making mistakes or overlooking minor details will often have fatal consequences, especially in the most dangerous chambers inside the keep. Your best and only weapon is your wits. Plan your moves carefully and remember to save your game often.

TRY AGAIN...

A Puzzle Sample (how to escape from the dungeon)

1 Pick yourself up off the floor and examine your surroundings. Find out more information by talking to the prisoner in the next cell.

2 The guard arrives with a meal. Eat it and take the bone. Examine the straw bed. Why is the straw cold and damp?

3 Pushing away the straw, you uncover a forgotten metal grate. It is too heavy to lift with just your fingers. If only you had a lever of some sort.

4 You select the bone and use it on the grate on the floor.

5 You pry the grate up with the bone! You can hear the water and smell the stench from the sewer below, but it beats staying in your cell. Time to make your escape!

14 15

Manual scan from archive.org/details/N64Manuals

With a game like Shadowgate, spoilers can especially compromise the experience, so we'll try to keep it as light as possible.
That said, even the manual for Shadowgate 64 ruined the first few puzzles!

How does the game track item status?

	1	in inventory
	3	not in inventory, placed in world
	4	not in inventory
	22	in inventory, equipped

The game maintains two separate lists in memory.
One is a fixed-sized array that stores a “status” for each item at its ID’s position.
This status is updated when an item is picked up, used, placed in the world, or equipped.

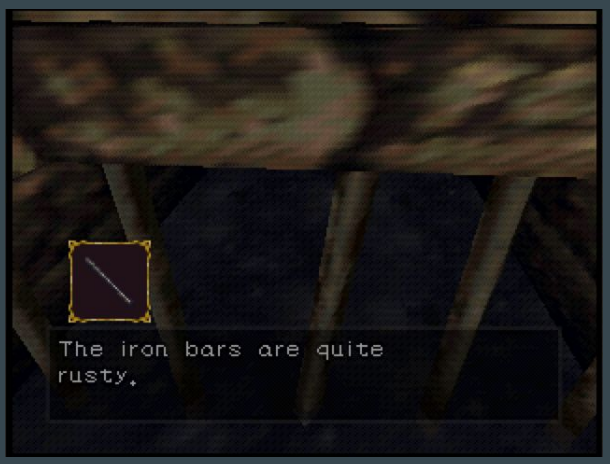
How does the game track item status?

The other is a variable-sized array that stores the ID’s of the current inventory, based on pick-up order.
Items in this list are added to the first position when picked up and removed when used.

Look-Up Table Concept

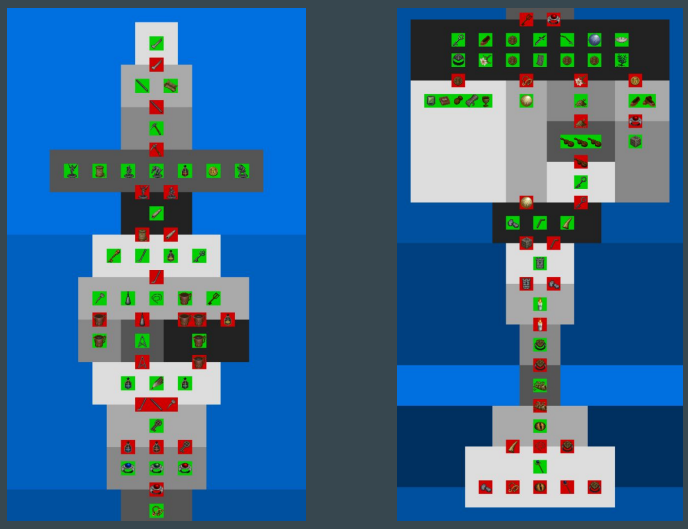
The core concept of this randomizer is a look-up table that maps the original ID’s to shuffled ID’s.

Player Experience Examples



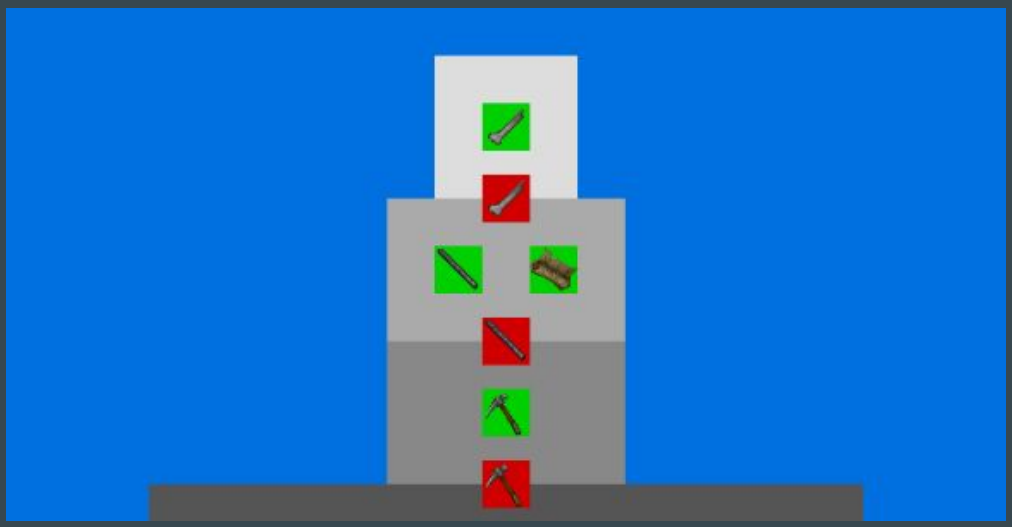
This randomizer doesn’t affect the 3D game world. Doing so would require much more knowledge of the game data.
Instead, the icons are now our source of truth while playing.
Some items will appear in their original locations, like the Iron Bar on the left, and others will be completely different, like the Dragon Flute on the right.

Game Design: Dependency Graph



In order to visualize the game logic, we can create a dependency graph.
Here is every item available in Shadowgate 64, along with which items gate progress.
When there are multiple paths available to the player, the gameplay becomes more complex, and the dependency graph becomes wider in those areas.

Game Design: Dependency Graph



Fortunately, this is the dependency graph for escaping the dungeon and sewers at the beginning of the game.
Typically you want your game design to remain “narrow” while introducing players to your game.
However, these narrow segments also limit our ability to randomize the experience.

Code: Dependency Graph

```
typedef enum {
    MAP                         = 0x2a,
    BONE                        = 0x2b,
    STAR_CREST                  = 0x2c,
    IRON_BAR                    = 0x2d,
```

```
#define ITEMS_DUNGEON BONE
#define ITEMS_SEWERS_1 MAP, IRON_BAR
```

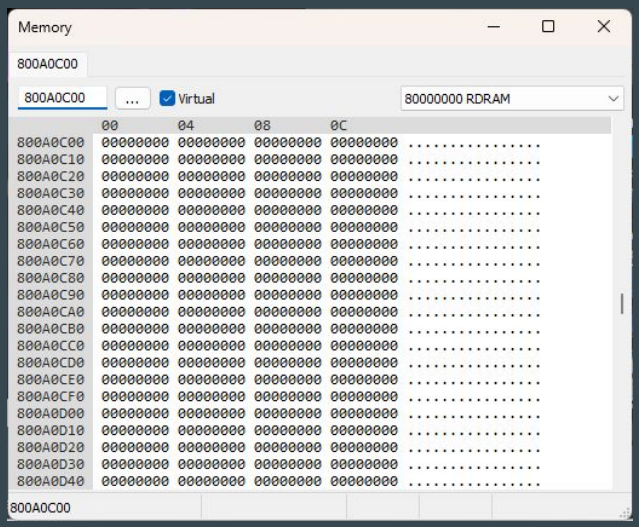
```
{
    BONE,
    {
        ITEMS_DUNGEON,
    },
},
```

```
{
    MAP,
    {
        EVERYTHING,
    },
},
```

```
{
    IRON_BAR,
    {
        ITEMS_DUNGEON,
        ITEMS_SEWERS_1,
    },
},
```

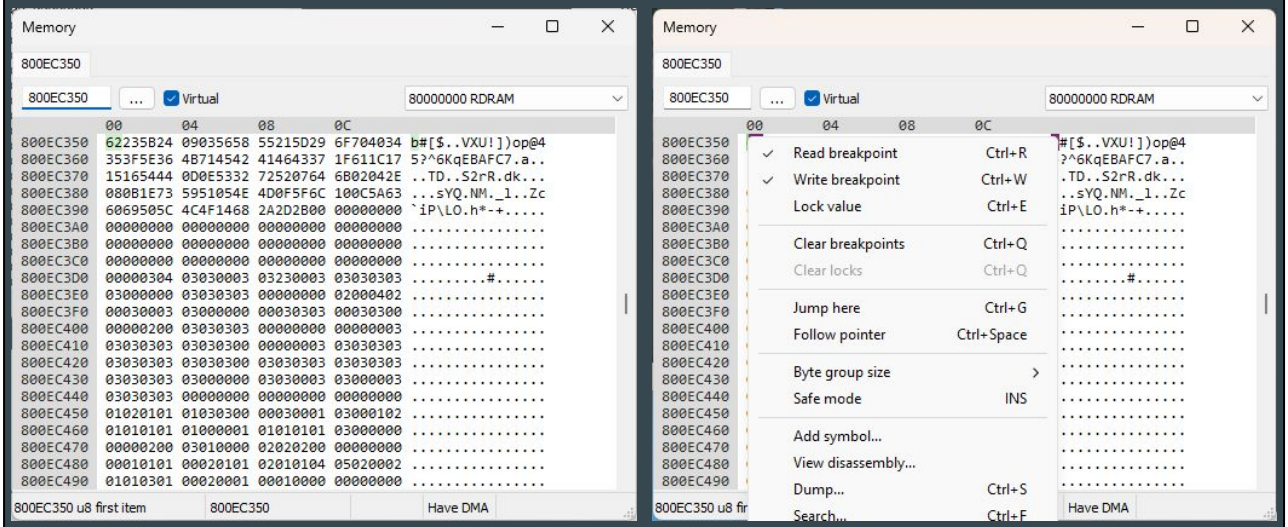
Here are some examples of how I represent the items, segments, and possible destinations for items when shuffling.
Initially, the player only has access to the Bone, so the Bone MUST stay in its original location.
The Map, while useful, is not required to beat the game, so this item may be shuffled to any location.
The Iron Bar is required to exit the first part of the sewers, so it must be placed in that area.

Finding Available Space in the ROM



We need to find unused space in the ROM to store our new logic and look-up table. Generally, this can be found simply by visually scanning for an appropriate segment of 0's or FF's towards the end of the ROM.
Project64 is an N64 emulator with many useful tools that we will use extensively in this project.
Here is the memory view of the Project64 emulator, showing the segment we will target.

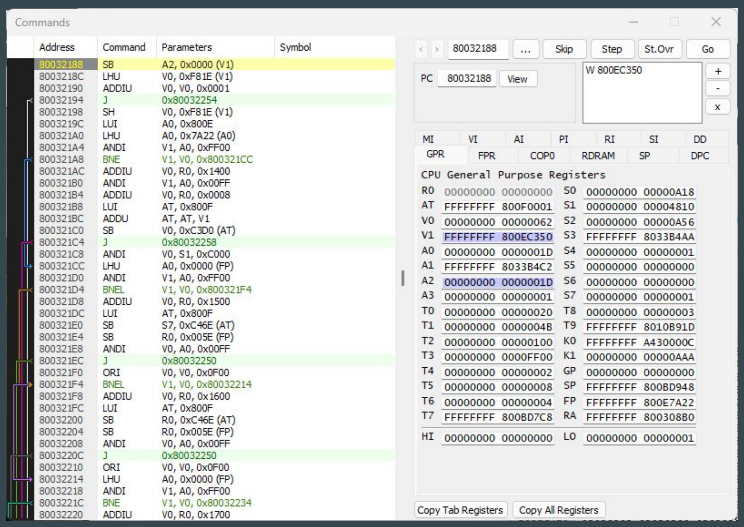
Finding Code That References the Inventory



We now need to locate existing code that uses the inventory in some way.
Finding this memory location can be done manually, but with most games, I would suggest checking for existing Game Genie, GameShark, or Action Replay codes to give you a head start.
There were plenty of us snooping through game memory in the 90's…

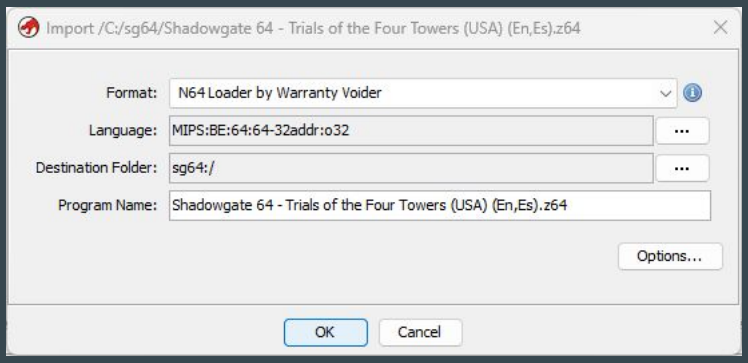
Once you find these locations, Project64 can be used to set read and write breakpoints.

Finding Code That References the Inventory



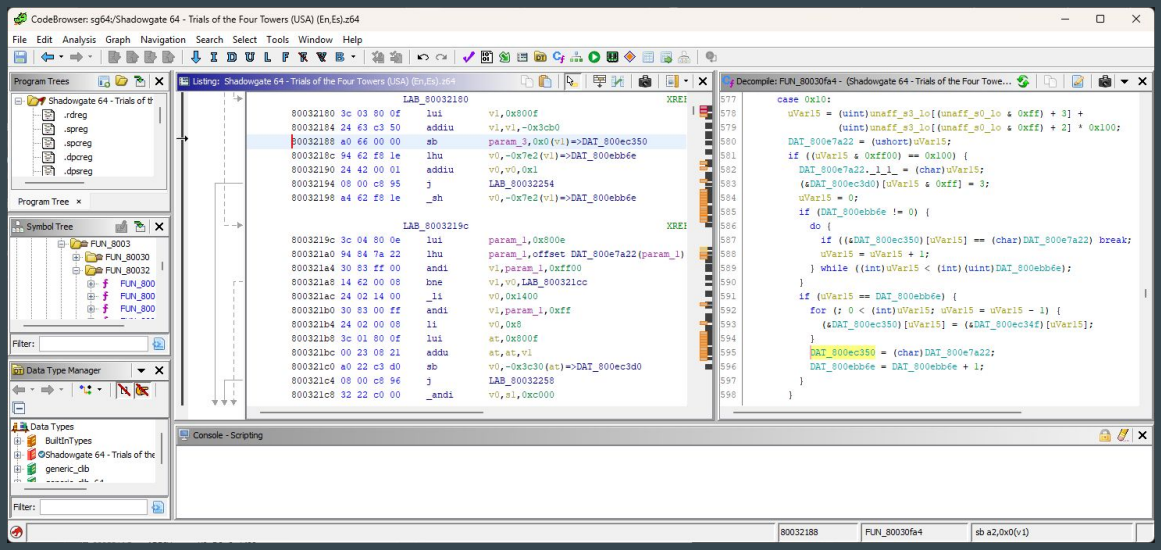
Here is an example of a write breakpoint being triggered in Project64.
You can see the assembly instructions on the left, and the registers contents on the right.
Playing through enough of the game in the emulator will allow you to make a list of all of these code locations.

Finding Code That References the Inventory



While it's entirely possible and often necessary to read the disassembled code, sometimes it's challenging to get the bigger picture from this output.
Ghidra is a software reverse engineering framework created and maintained by the NSA which provides a powerful, free way to decompile this code into C, which can be much easier to read and understand.
Warranty Voider on github has created many loaders for retro platforms, including the N64, which help with the initial setup and decompilation.

Finding Code That References the Inventory



Here you can see that same disassembly we saw in Project64 on the left, along with the decompiled C code on the right.
It's now much easier to tell that this code is responsible for moving each item ID down one position in the inventory array at 800ec350 (line 592), storing the new item ID at the front of that array (line 595), and incrementing the inventory size variable (line 596).

Cleaning Up Ghidra's Decompilation

```
void FUN_80030dd0(uint param_1) {
  uint uVar1;

  if ((param_1 & 0xff00) == 0) {
    *(undefined1 *)((param_1 & 0xff) + 0x800ec3d0) = 0xff;
  } else if ((param_1 & 0xff00) == 0x4000) {
    *(undefined1 *)((param_1 & 0xff) + 0x800ec3d0) = 0;
  } else if ((param_1 & 0x8000) == 0) {
    *(byte *)((param_1 & 0xff) + 0x800ec3d0) = (byte)((param_1 & 0xff00) >> 8) |
      *(byte *)((param_1 & 0xff) + 0x800ec3d0);
  } else {
    *(byte *)((param_1 & 0xff) + 0x800ec3d0) = *(byte *)((param_1 & 0xff) +
      0x800ec3d0) & (byte)(0xff00 - (param_1 & 0x7f00) >> 8);
  }
// ...
```

That said, Ghidra's output isn't always the easiest to read.
Here is another example of some code we need to trace, and at first glance it's quite opaque.

Cleaning Up Ghidra's Decompilation

```
void updateItemStatus(uint16_t itemCode) {
  uint8_t itemId = itemCode & 0xffu;
  uint8_t itemAction = (itemCode & 0xff00u) >> 8;

  if (itemAction == SET_ALL) {
    inventoryStatus[itemId] = 0xffu;
  } else if (itemAction == CLEAR_ALL) {
    inventoryStatus[itemId] = 0u;
  } else if ((itemAction & STATUS_FLAG_CLEAR_ACTIONS) == 0u) {
    inventoryStatus[itemId] |= itemAction; // set bits in itemAction
  } else {
    // clear bits in itemAction
    inventoryStatus[itemId] &= ~(itemAction & 0x7fu);
  }
// ...
```

However, I would encourage you to manually re-write these code segments as you explore and experiment.
Reverse Engineering is just guesswork until you start making progress!

Identifying All References

- Override the item icon displayed.
- Override the in-world item pick-up.
- Override the item status update.
  - Example: Jezibel gives you her "Pendant."
- Override the item status check.
  - Example: Jezibel will only release the "Family Diary" once you have received her "Pendant."

With all of these approaches, we can identify four code paths for referencing the inventory which will need to be adjusted for our randomizer.

The Best Laid Plans

- Override the item icon displayed.
  - **Exception: don't shuffle when handing items to Agaar.**
- Override the in-world item pick-up.
- Override the item status update.
  - Example: Jezibel gives you her "Pendant."
  - **Exception: don't shuffle when clearing item status.**
  - **Exception: don't shuffle when returning items to player.**
- Override the item status check.
  - Example: Jezibel will only release the "Family Diary" once you have received her "Pendant."
  - **Exception: don't shuffle "is equipped" checks.**
  - **Exception: don't shuffle when checking event triggers.**

However, you know what they say about the best laid plans.
Through experimentation, we also can identify cases where we DON'T want to apply our shuffling logic.

Updated Logic Based on Item Action

- Override the item icon displayed.
  - Exception: don’t shuffle when handing items to Agaar.
- Override the in-world item pick-up.
- Override the item status update.
  - Example: Jezibel gives you her "Pendant."
  - **Exception: don’t shuffle when clearing item status.**
  - Exception: don’t shuffle when returning items to player.
- Override the item status check.
  - Example: Jezibel will only release the "Family Diary" once you have received her "Pendant."
  - **Exception: don’t shuffle “is equipped” checks.**
  - Exception: don’t shuffle when checking event triggers.

These exceptions are easily handled by checking what status is being applied to the item.

We Still Need a Good Way to Handle These

- Override the item icon displayed.
  - **Exception: don't shuffle when handing items to Agaar.**
- Override the in-world item pick-up.
- Override the item status update.
  - Example: Jezibel gives you her "Pendant."
  - Exception: don't shuffle when clearing item status.
  - **Exception: don't shuffle when returning items to player.**
- Override the item status check.
  - Example: Jezibel will only release the "Family Diary" once you have received her "Pendant."
  - Exception: don't shuffle "is equipped" checks.
  - **Exception: don't shuffle when checking event triggers.**

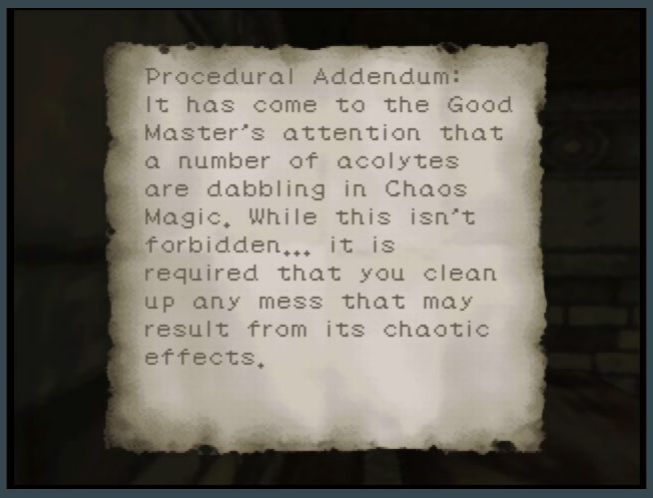
These, however… oh no. Does anyone have any ideas?
[Check the current room ID.]
This was the solution that unlocked the rest of the logic, and was astutely suggested by my wife.

Strategy for Changing as Little as Possible

- For each of these references, jump to our borrowed code space.
- If our item action and room ID checks pass, override the item ID by looking up the new item in the shuffled look-up table.
- Minimize registers modified in new code for safety.
- Avoid modifying the program stack!

So here’s the plan.

Strategy for Changing as Little as Possible



I think this note, which appears in-game, is a great summary.

Pop Quiz!

- Most N64 games execute 64-bit instructions.
  - True
  - False

Pop Quiz!

- Most N64 games execute 64-bit instructions.
  - True
  - **False**

While Nintendo marketing would have you believe otherwise, most games execute 32-bit instructions on the MIPS processor.
This is due to one simple fact: 64-bit instructions take twice as much space on the cart!
Because of this, we'll see examples of multiple 32-bit instructions being used to construct 64-bit addresses.

ROM Patch Example #1

```
// default item pickup path
replaceCommand(rom, 0x800320e8u, 0x0802832cu); // J 0x800a0cb0

replaceCommand(rom, 0x800a0cb0u, 0x3c01800au); // LUI AT, 0x800a
replaceCommand(rom, 0x800a0cb4u, 0x00260821u); // ADDU AT, AT, A2
replaceCommand(rom, 0x800a0cb8u, 0x90260C00u); // LBU A2, 0x0c00 (AT)
replaceCommand(rom, 0x800a0cbcu, 0x0800c83bu); // J 0x800320ec
replaceCommand(rom, 0x800a0cc0u, 0x30C200ffu); // ANDI V0, A2, 0x00ff
```

Here is a segment of the actual randomizer code.
On each line, we are overriding the value at a specific location with 4 bytes (32 bits) of new machine code (the result of compiling the human-readable code) as well as the assembly equivalent in a comment for convenience.
This code is hijacking execution at a specific instruction into our new code, then constructing an address inside our look-up table based on A2.
AT = 0x800a0c00 (look-up table) + A2 (item ID)
A2 = value at AT
V0 = A2, for future code

ROM Patch Example #2

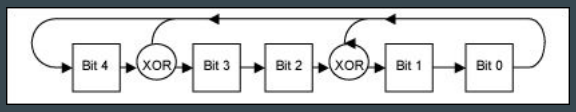
```
// update item display
// prevent item look-up in room 0x01 (Dungeon)
replaceCommand(rom, 0x80030cc0u, 0x0c028320u); // JAL 0x800a0c80
replaceCommand(rom, 0x80030cc4u, 0x00000000u); // NOP

replaceCommand(rom, 0x800a0c80u, 0x3c01800eu); // LUI AT, 0x800e
replaceCommand(rom, 0x800a0c84u, 0x9421a726u); // LHU AT, 0xa726 (AT)
replaceCommand(rom, 0x800a0c88u, 0x24070001u); // ADDIU A3, R0, 0x0001
replaceCommand(rom, 0x800a0c8cu, 0x10270006u); // BEQ AT, A3, 0x800a0ca8
replaceCommand(rom, 0x800a0c90u, 0x30a700ffu); // ANDI A3, A1, 0x00ff
replaceCommand(rom, 0x800a0c94u, 0x30a5ff00u); // ANDI A1, A1, 0xff00
replaceCommand(rom, 0x800a0c98u, 0x3c01800au); // LUI AT, 0x800a
replaceCommand(rom, 0x800a0c9cu, 0x00270821u); // ADDU AT, AT, A3
replaceCommand(rom, 0x800a0ca0u, 0x90270c00u); // LBU A3, 0x0c00 (AT)
// ...
```

Here is another example, this time using Jump And Link instead of Jump, because the stack was already setup for a function call at this code location.
We first construct the address 0x800ea726, the location of the current room ID.
We then compare that room ID to 1. Since MIPS does not have an instruction to load an immediate byte, we add 1 with register R0, which always contains zero.
If these values are equal, we are in room ID 1, and we skip past the logic to apply our look-up table.
The rest of the logic is similar to example #1.

The Importance of Deterministic Seeds

- Linear Feedback Shift Registers to the rescue!
- 32-bit Input:
  - 0xdecafbad
- 8-bit Output:
  - 138
  - 69
  - 254
  - 127
  - 99



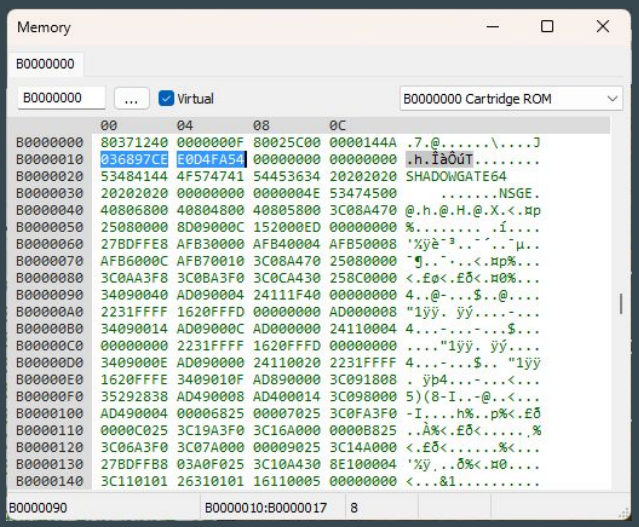
LFSR logic from analog.com

When a player creates a ROM with a particular seed value, it's important that that seed always result in the same shuffling.
This will enable things like easier debugging and sharing of particularly interesting experiences.
We will accomplish this by using a Linear Feedback Shift Register to generate all random values while shuffling.
LFSRs use polynomial "taps" to modify new values based on old values while shifting.
LFSRs will always produce the same sequence of values for the same input seed, which is exactly what we want.

Limitations

- Certain items will never be shuffled, to avoid soft-locks or hangs.
- Certain items locations will never hold critical-path items, to avoid losing them via sequence breaks.

Additional Considerations: Header Checksums



Bytes 16 - 19 and 20 - 23 of every N64 game header contain two four-byte checksums based on the first 1MB of the cartridge.
Each game uses one of five different checksum algorithms, the most common of which is CIC-6102.
The CIC security chip confirms the checksums during Initial Program Load 3, the final stage of the console's built-in boot code, and will refuse to boot the game if there is a mismatch.
Most emulators will ignore this discrepancy.
When modifying a ROM, you should always update the checksums for maximum compatibility.

Pop Quiz!

- Accessing a game cartridge is much faster than accessing a CD.
  - True
  - False

Pop Quiz!

- Accessing a game cartridge is much faster than accessing a CD.
  - **True**
  - False

This is why load screens are much more prevalent on the Playstation compared to the N64.

Pop Quiz!

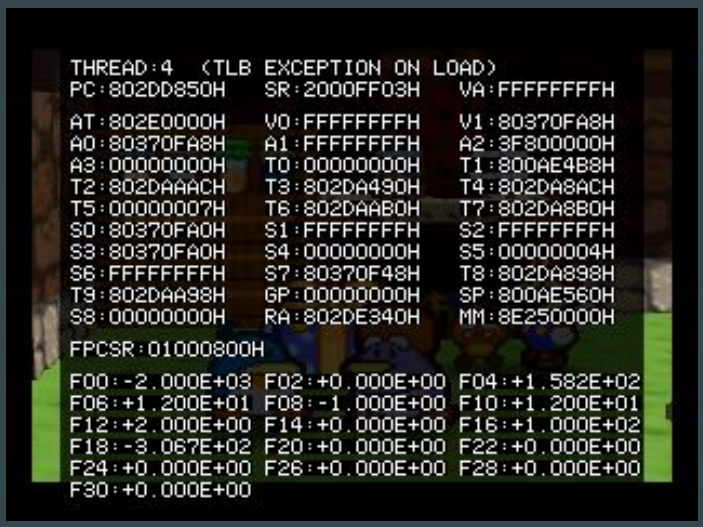
- Most N64 games execute code straight from the game cartridge.
  - True
  - False

Pop Quiz!

- Most N64 games execute code straight from the game cartridge.
  - True
  - **False**

It’s actually still faster to read from RAM, so most N64 games will copy code into memory before executing it.

Add’l Considerations: Translation Lookaside Buffer

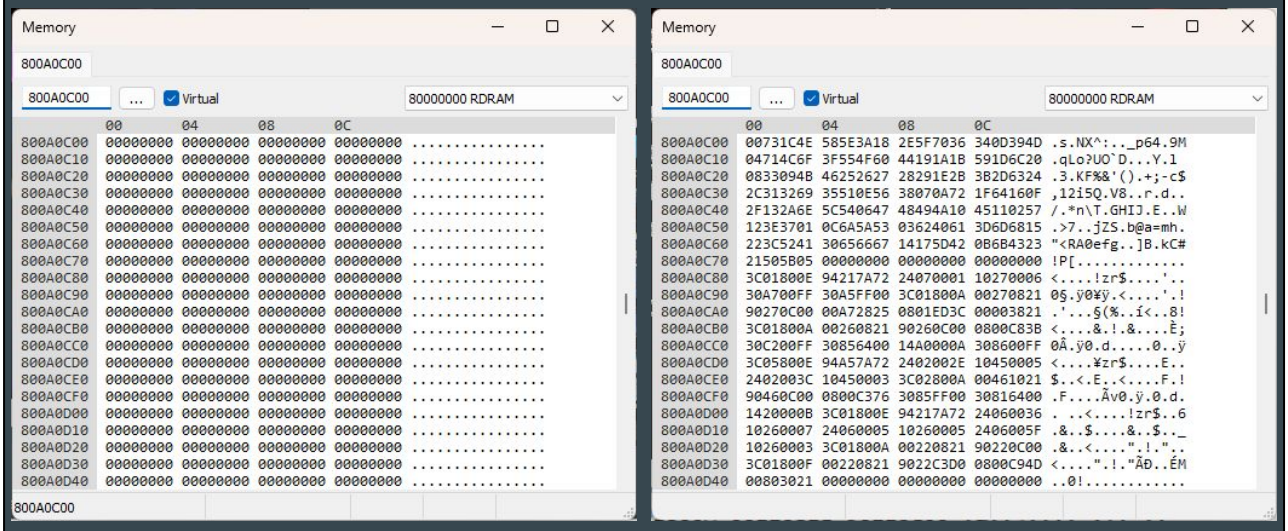


Paper Mario 64 crash handler screenshot by A gossip-loving Toad on mariowiki.com

Large copies from cartridge to RAM are typically handled by the Translation Lookaside Buffer system on the N64.
This system is responsible for mapping virtual memory operations to physical memory locations.
Here’s the bad news: Shadowgate 64 only performs one TLB load at boot, which makes it easy to work with, but your favorite game might do this quite often.
Viper182 said it best: “TLB mapping is pretty evil in the eyes of ASM hackers. This is the reason you don't see 1337 ASM codes for Goldeneye and very few for Perfect Dark.”

The screenshot is an example of a virtual memory request in Paper Mario 64 that cannot be accommodated by the TLB, so the game just crashes.

Putting It All Together



Putting all of this together, we're finally ready to fill in our borrowed ROM space!

Questions, Suggestions, and/or Gratuitous Praise?

- Grab us after the talk (even if it's in the parking lot) and let's chat about any questions you might have!

- @chimbraca and @SleepyBitsNES on Twitter
- @grendell and @SleepyBitsGames on Bluesky
- @SleepyBitsGames on Instagram
- @SleepyBitsGames on YouTube for game teasers and trailers

- feedback@sleepybitsgames.com

Thank you!!